
Big Endian vs. Little Endian (for project 1)

- Memory can be thought as one large array containing bytes
- Use “address” to refer to the array location
- Each address stores one element, which is typically one byte (byte-addressable)
- To store 32-bit integer, like 5, we need 4 bytes so 4 slots of RAM array of bytes. Should the 00000101 byte be at the first or fourth byte of the 4 bytes?
- Big Endian: stores the leftmost significant byte in the lowest numerical byte address, so 00000101 is in the 4th byte
- Little Endian: stores the rightmost significant byte in the lowest numerical byte address, so 00000101 is in the 1st byte

Integer: 0x0A0B0C0D:

Big Endian

Addresss	100	101	102	103
Value	0A	0B	0C	0D

Little Endian

Addresss	100	101	102	103
Value	0D	0C	0B	0A

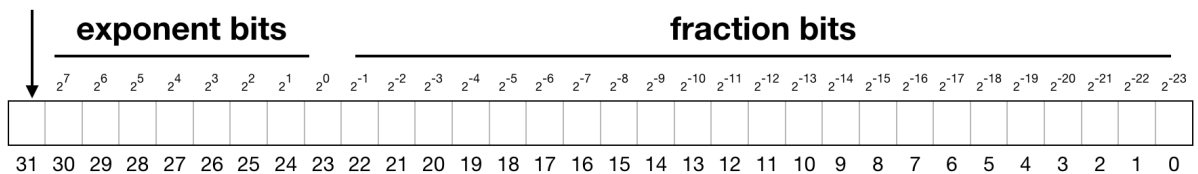
Floating Point Format

Floating point numbers are represented in the IEEE-754 format. A single precision float is represented by 32 bits. These 32 bits are split up into 3 parts - the sign bit, the exponent, and the fraction. The number being represented is computed according to

$$\pm 2^e \times \text{fraction}$$

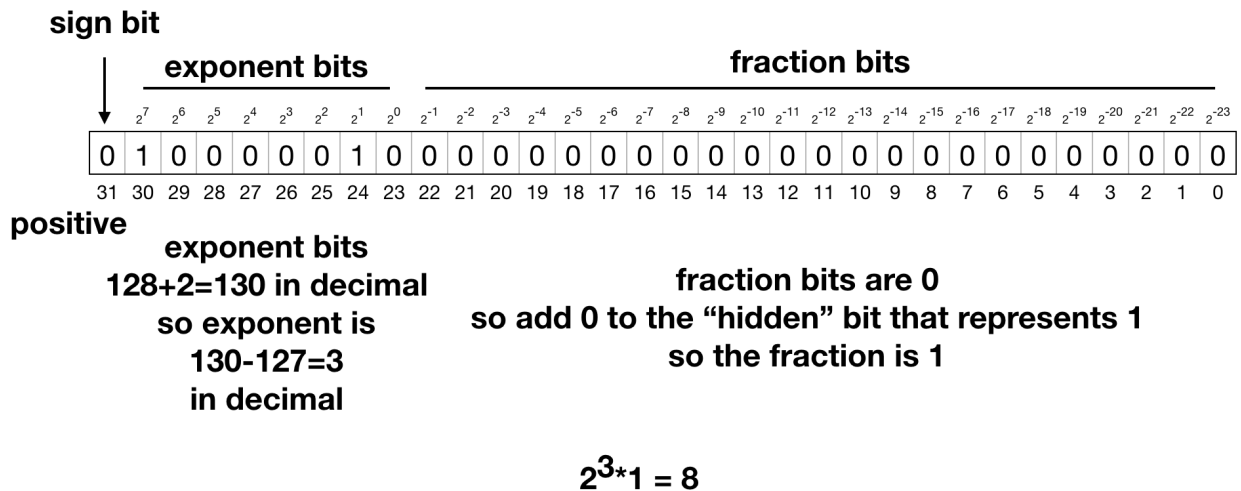
- The leftmost bit (bit 31) is the sign bit (1 indicates negative, 0 indicates positive).
- The next 8 bits (bits 30 through 23) are used as an unsigned 8-bit integer (let's call that `exp_bits`) and contain the exponent. $e = \text{exp_bits} - 127$.
- The remaining 23 bits (bits 22 through 0) represent the fraction, also known as the mantissa or significand. This represents a number between 1 and 2 (it approaches 2). Even though we have only 23 bits, we treat it as a 24-bit number with the first bit always as a 1. That first ("hidden") left-most bit represents $2^0 = 1$. The next bit represents $2^{-1} = 1/2$, the next is $2^{-2} = 1/4$, ..., and the last bit is 2^{-23} . We add the value for all bits with 1's in them. I.e. if the 2^{-1} bit has a 1 in it, then we add 1 (because the hidden bit is always 1) to $2^{-1} = 1/2$ to get 1.5. The fraction is 1.5.

sign bit

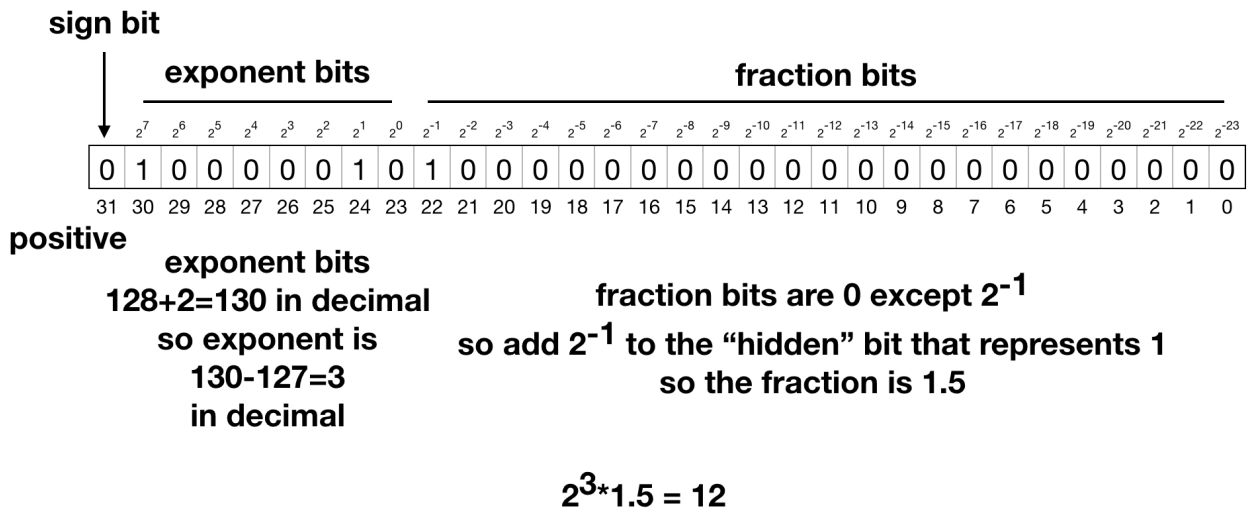


- Check out this website that shows how floating point numbers are represented. You can edit either the decimal representation or the binary representation: <https://www.h-schmidt.net/FloatConverter/IEEE754.html>

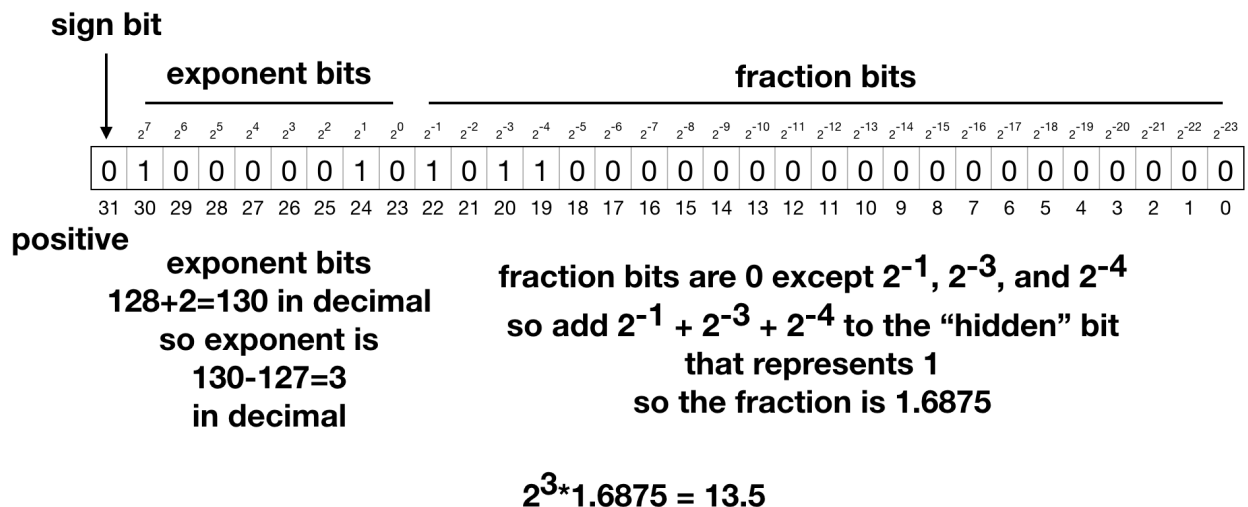
- **Example 1:** Look at the floating point representation of 8.



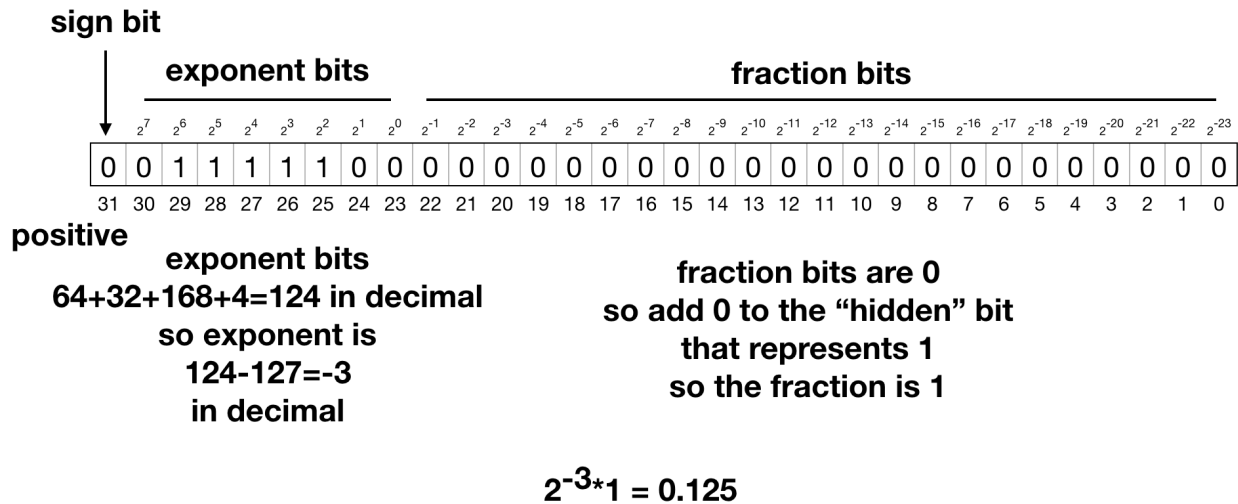
- **Example 2:** Look at the floating point representation of 12.0



- **Example 3:** Look at the floating point representation of 13.5



- **Example 4:** Look at the floating point representation of 0.125 (i.e. $1/8$)



- Very small numbers (2^{-127}) can be represented, as can very large numbers (2^{128}).
- But when we perform arithmetic with them, we are constrained. What happens when we add two numbers of different orders of magnitude (i.e. different exponents)? Think about how you would do that by hand - you would represent both numbers with the same exponent. And, if you have a fixed number of digits to represent the number, you would lose information when you represent your smaller number with the larger exponent. The computer is forced to follow a similar process, so we lose information - we call this round-off error.
- There is a trade-off between the range of the numbers that can be represented (more bits in the exponent gives us a wider range) and the precision (more bits in the fraction allows us to be more precise in representing the number).